

Project architecture & progress report

What is built, how it works,
and what comes next.

Current stage: local backend MVP

The full PDF-to-answer pipeline is implemented. Automated code checks pass in the default test mode. Reliable grounded answers and deployment readiness remain unfinished.

69 tracked files

54 Python files reviewed

5 application endpoints

Upload, search, chat, health, ready

97 tests passed

216 subtests passed; 18 tests skipped

The project in one paragraph

AI Chatbot is a Python service that turns searchable PDFs into a knowledge base. It extracts text by page, splits text into smaller passages, stores their numerical representations in PostgreSQL, and retrieves relevant passages for a language model to answer questions with document and page references.

Snapshot and review boundary

Reviewed the working tree at commit **c916486** (30 September 2026), including the existing uncommitted edit to **.env.example**. The application version is **0.1.0**. This report was prepared on **1 October 2026**, Asia/Kolkata.

All tracked project files were reviewed, including empty scaffolds, tests, migrations, and dependency metadata. Generated folders were classified; third-party packages and Git object storage were not audited line by line. Secret values from **.env** are omitted.

Evidence: Source inspection; git ls-files and git log; current pytest run. Full inventory: pages 14-17.

How far is it complete?

The core backend workflow is built. Its current acceptance gap is dependable answers on real documents, followed by the work required to make the service usable beyond a local developer setup.

Area	Current state	Meaning
Foundation & database	Implemented	Startup lifecycle, async storage, migration, health and readiness.
PDF ingestion	Implemented	Validation, extraction, chunking, embeddings, deduplication and retry.
Vector retrieval	Implemented	Exact cosine search with optional document selection.
RAG chat & citations	Implemented	Generation, source references, fallback and provider-error mapping.
Tests & diagnostics	Implemented; partial live verification	Default suite passes today. Saved real-model evaluation has unresolved failures.
Reliable policy answers	Open	Two of three saved live evaluation cases failed.
Product interface	Absent	API docs and CLI exist; no dedicated chat frontend or conversation history.
Shared deployment	Unfinished	No auth/ownership checks, app container, job recovery or CI pipeline.

Why there is no completion percentage

The repository has no agreed full-product scope or weighted acceptance checklist. A single percentage would hide the difference between implemented code, model quality, and production readiness.

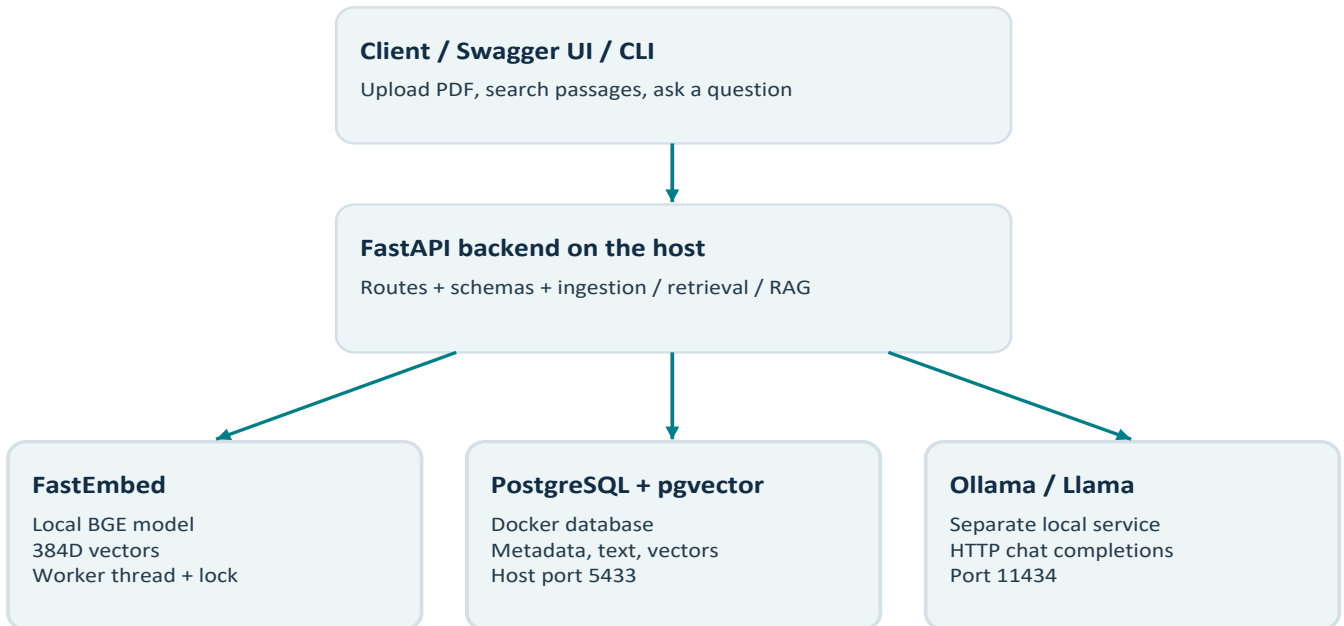
Reading guide

Pages	What you will learn
3-6	Architecture, ingestion, chat flow and database design
7-9	Technology versions, configuration and API specifications
10-13	Verification, limitations, milestones and local operation
14-17	File-by-file inventory and generated/local folder classification

Evidence: app/; tests/; Dockerfile; compose.yaml; MVP_REVIEW.md; saved policy-evaluation traces.

System architecture

A layered Python application runs on the host machine. PostgreSQL runs in Docker, and Ollama is a separate service. FastEmbed loads inside the application process.



Responsibilities by layer

Layer	Responsibility
API + schemas	FastAPI routes validate input/output with Pydantic and translate failures into HTTP responses.
Services	Coordinate extraction, chunking, retrieval, prompt construction, generation and citation validation.
Providers	Parse PDFs and provide an embedding interface. The factory currently supports FastEmbed with one fixed BGE model.
Storage	Own SQLAlchemy engines, sessions, document claims, atomic writes and pgvector search.

Important design choices

Short database work: ingestion performs expensive parsing and inference outside database transactions. Chat ends its retrieval transaction before awaiting model generation.

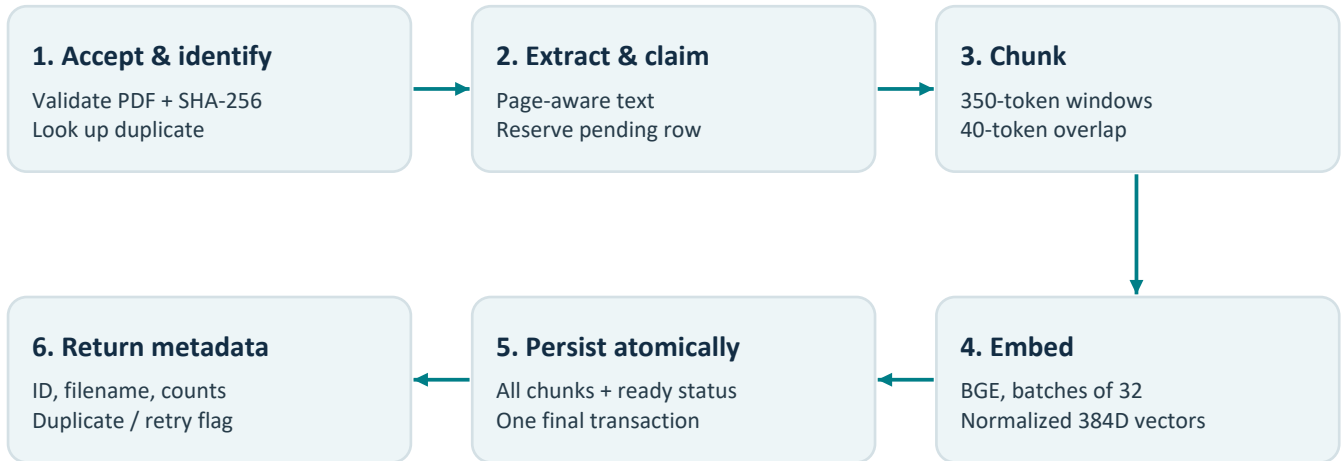
Explicit integration: the code coordinates RAG directly; no LangChain or LlamaIndex orchestration dependency is declared. The embedding abstraction is replaceable in design, but the database and adapter currently enforce one embedding space.

Local topology: run.py binds the API to 127.0.0.1:8000. Compose publishes only PostgreSQL on localhost:5433. An alternate configured LLM URL sends questions and retrieved text to that provider.

Evidence: run.py; app/main.py; app/api/v1/router.py; app/providers/base.py; app/storage/db.py; compose.yaml.

PDF ingestion flow

Ingestion means converting an uploaded file into searchable text and vectors. The HTTP endpoint and the ingestion CLI share the same service.



Specification	Current implementation
Accepted input	application/pdf with a safe .pdf filename; maximum 255 filename characters.
File / content limits	10 MiB; 100 pages; 2,000,000 extracted characters; 1,000 chunks per document.
Text extraction	pypdf strict parsing; one-based page numbers; blank pages skipped without renumbering. NUL characters become spaces.
Chunking	350 tokens with 40-token overlap inside each page. Original text slices are preserved; encoded chunks must fit a 480-token safety limit.
Embedding / writing	Batches of 32 for inference; at most 256 rows per write batch. All final writes are committed together.

Document state and repeated uploads

New valid PDF: pending -> ready on success, or pending -> failed on ordinary processing failure. Invalid extraction is rejected before reserving a database row.

Same bytes: SHA-256 uniqueness prevents duplicate claims. Ready duplicates return the original ID and filename. Failed documents can retry with that same identity. A pending duplicate returns HTTP 409 and Retry-After: 5.

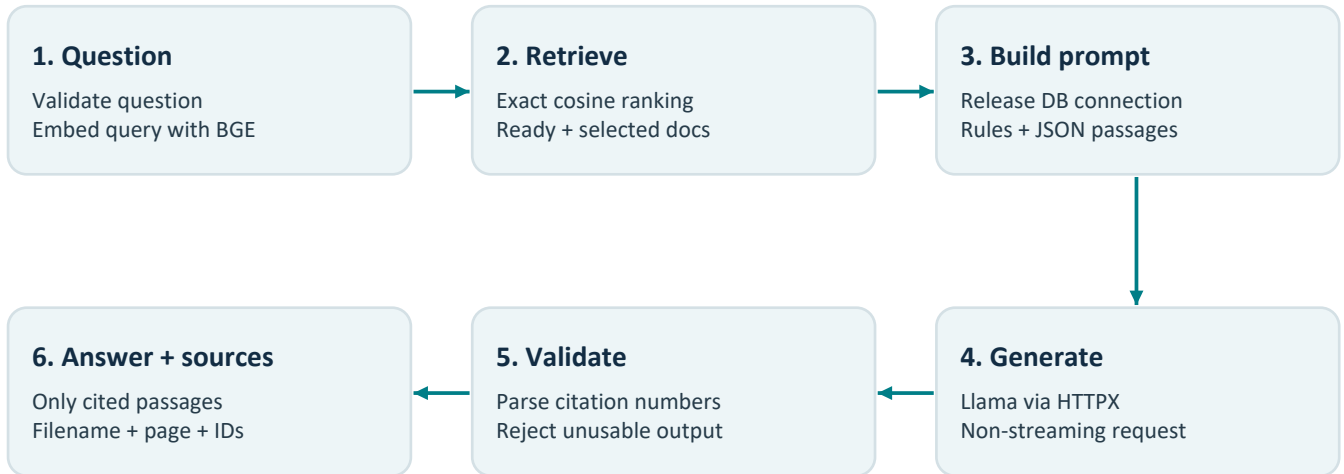
Boundaries

Scanned/textless and encrypted PDFs are rejected; OCR is absent. Original PDF bytes are not retained by ingestion. A hard process kill can leave a pending claim requiring operator recovery.

Evidence: `app/providers/documents.py`; `app/services/{ingestion,chunking,document_hash}.py`; `app/storage/documents.py`.

Search and answer flow

Retrieval-augmented generation (RAG) gives the language model relevant document passages alongside the question. Embeddings find passages; the Llama model writes the answer.



What search does

The question becomes a normalized 384-dimensional BGE vector. SQL joins chunks to documents, selects only ready documents and the supported embedding space, and ranks by ascending cosine distance. Equal distances use chunk UUID order as a stable tie-breaker.

The default **top_k** is 5; allowed values are 1-20. Optional **document_ids** selects 1-20 UUIDs before ranking and limiting. An unknown ID returns no matches. Search uses exact vector ranking; no approximate vector index, keyword hybrid search, reranker or relevance cutoff is implemented.

How generation is controlled

Trusted grounding rules are sent in a system message. A user message carries JSON-encoded passages and the question. HTTPX sends a non-streaming chat completion request with temperature 0 and a 1,024-token output limit. There is no automatic LLM retry.

Empty retrieval returns the canonical insufficient-context answer without calling the model. A recognized model fallback also returns no sources. Otherwise the service checks citation syntax and range, keeps only cited sources, and rennumbers citations to match the response list.

References are checked; factual support still needs evaluation

A valid citation number does not prove that its passage supports a claim. The code does not score entailment or enforce a generation-token budget for the full prompt. Prompt instructions also do not guarantee resistance to instructions embedded in documents.

Evidence: `app/services/{retrieval,rag,llm}.py`; `app/prompts/rag.py`; `app/storage/documents.py`.

Database and persistence

PostgreSQL stores both relational metadata and vector embeddings. One document has many chunks. Alembic owns schema changes.

documents	Type / rule
id	UUID primary key
filename	String(255), required
content_sha256	String(64), required and unique; digest computed from exact file bytes
page_count	Integer greater than zero
status	pending, ready or failed; server default pending
created_at	Timezone-aware timestamp; server default now()

document_chunks	Type / rule
id / document_id	UUID primary key / foreign key to documents.id with cascading deletion
chunk_index / page_number	Zero-based index / one-based page; both constrained to valid lower bounds
content	Required nonblank text
embedding_space	Fixed value: fastembed:bge-small-en-v1.5:384:l2:v1
embedding	Required pgvector VECTOR(384)
Unique position	(document_id, chunk_index) must be unique

Integrity and transaction boundaries

The database enforces types, foreign keys, uniqueness and basic checks. The ingestion service additionally validates finite, normalized vectors; Pydantic checks duplicate chunk indexes and page bounds within a batch. These application checks are separate from SQL constraints.

The initial migration is **7f7483ec3680**. The Docker initialization SQL enables the vector extension on a fresh database volume. Startup checks connectivity and the extension, but does not apply migrations or confirm the deployed revision.

What is retained

Metadata, extracted chunk text and vectors are stored. Original PDFs, chat sessions and message history have no storage model here. Stored text can be embedded again; fresh extraction and faithful re-chunking require the original file.

Evidence: app/storage/{models,documents,db}.py; migrations/versions/7f7483ec3680_create_documents_and_document_chunks.py; infra/postgres/init/001-vector.sql.

Technology and versions

Versions below are the installed project environment observed during this review, not claims about the latest available releases. pyproject.toml declares ranges; uv.lock records the dependency graph.

Technology	Observed version	Role
Python	3.12.14	Runtime; project requires ≥ 3.12 ; .python-version selects 3.12.
FastAPI / Uvicorn	0.141.1 / 0.53.0	HTTP API, OpenAPI documentation and ASGI serving.
Pydantic / pydantic-settings	2.13.5 / 2.15.0	Typed contracts and environment configuration.
SQLAlchemy / Psycopg	2.0.54 / 3.3.6	Async database layer and PostgreSQL driver.
Alembic	1.20.0	Database migration management.
pgvector Python package	0.5.0	Vector SQLAlchemy type and cosine-distance expressions.
FastEmbed / ONNX Runtime	0.8.1 / 1.30.0	Local embedding inference; ONNX Runtime is transitive.
Hugging Face Hub / tokenizers	1.33.0 / 0.23.2	Tokenizer assets and token-aware text windows.
pypdf / python-multipart	6.19.0 / 0.0.32	PDF text extraction and multipart upload parsing.
HTTPX	0.28.1	Async LLM HTTP client and test transports.
pytest	9.1.1	Test runner; tests also use unittest, mocks and AnyIO support.

Models and infrastructure

Embedding model: BAAI/bge-small-en-v1.5; 384 dimensions, L2 normalization, separate query and passage embedding methods. Model loading is lazy and cached; inference is serialized with a threading lock.

Generation model: the code and example select llama3.2:1b-instruct-q4_K_M through an Ollama-compatible endpoint. This review did not verify a running model server or its installed version.

Database image: compose.yaml pins pgvector/pgvector:0.8.6-pg16-trixie, representing PostgreSQL 16 with pgvector. Docker was not running during this review, so the actual running database version was not checked.

Dependency footprint: 13 direct runtime dependencies, one declared development dependency, and 56 lockfile package records including the root project. No frontend package manifest is present.

Evidence: pyproject.toml; uv.lock; .python-version; installed distribution metadata; compose.yaml; app/services/llm.py.

Configuration and runtime limits

The application uses validated environment settings. Process environment variables take precedence over .env. Application/database settings are cached; LLM settings are loaded when preparing each request.

Setting group	Defaults / example values	Operational meaning
Application	APP_NAME=AI Chatbot APP_ENV=development	Environment is validated as development/test/production. No separate deployment profile is implemented.
Database address	127.0.0.1:5433	Required database name, username and password come from configuration; secret values are not reproduced here.
Database pool	5 connections; 0 overflow 5-second pool wait	Limits simultaneous checked-out connections per application process.
Database timeouts	5-second connect 5,000 ms statements 8-second health deadline	Bounds connection setup, SQL execution and readiness checks.
Embeddings	fastembed; BGE-small 384 dimensions; 2 threads	Provider/model/dimensions must match the fixed schema. Threads can be configured from 1 to 32.
Model endpoint	http://127.0.0.1:11434/v1	The client appends /chat/completions. Local example has an empty API key.
Generation	Temperature 0 1,024 output tokens 5-second connect	Non-streaming request; no automatic retry; partial output is rejected.

Timeout values currently differ

Location	LLM_TIMEOUT_SECONDS
app/services/llm.py fallback	30 seconds when unset
Current .env.example	60 seconds (existing user edit preserved)
README.md setup example	120 seconds
Saved 30 September policy traces	60 seconds effective at the time of those runs

Allowed generation deadlines are greater than zero and at most 600 seconds. The report does not treat the example file as proof of the current private configuration. Increasing a deadline can allow slow inference to finish, but does not establish answer accuracy.

Evidence: app/core/config.py; app/services/llm.py; .env.example; README.md; saved trace generation settings.

API specification

Five application endpoints are mounted under `/v1`. FastAPI also supplies interactive documentation and an OpenAPI schema. There is no authentication on the application routes.

Endpoint	Input	Successful output
GET <code>/v1/health</code>	None	status=ok and a health message. Checks that the handler can respond.
GET <code>/v1/ready</code>	None	status=ready when database access and vector extension check pass; otherwise 503.
POST <code>/v1/documents/upload</code>	Multipart field: file	document_id, filename, page_count, chunk_count, status=ready, already_existed. New: 201; duplicate/retry: 200.
POST <code>/v1/documents/search</code>	query; optional top_k and document_ids	query, top_k, matches, results. Results contain text, cosine_distance and document/page/chunk metadata.
POST <code>/v1/chat</code>	question; optional top_k and document_ids	answer and sources. Each source has document_id, chunk_id, filename and page_number.

Request contract

Chat questions are trimmed, nonblank strings of at most **500 characters**. Search queries are trimmed and nonblank but have **no explicit maximum length**. top_k must be a strict integer from 1 to 20, default 5. document_ids accepts 1-20 UUIDs or may be omitted. Undeclared fields are rejected.

Failure behavior

Operation	Important HTTP statuses
Upload	409 pending duplicate; 413 oversized; 415 wrong media type; 422 invalid PDF/metadata/limits; 503 dependency failure; 500 unexpected error.
Search	422 schema failure; 400 service parameter failure; 503 storage unavailable; 500 embedding or internal failure.
Chat	422 invalid request; 429 provider rate limit; 502 unusable provider output/citations; 503 LLM unavailable; 504 timeout; 500 retrieval/storage/internal failure.

Readiness does not check migrations, document tables, embedding model availability or LLM response quality. The HTTP error handling generally avoids returning provider details, database credentials or private prompts.

Evidence: `app/api/v1/*.py`; `app/schemas/{chat,documents,health,readiness}.py`.

What was actually verified?

Code behavior and language-model quality need separate evidence. This page distinguishes this review's checks from older results stored in the repository.

Evidence	Result	Scope
Current default suite 1 October 2026	97 passed 18 skipped 216 subtests passed	pytest with deprecation warnings treated as errors; completed in 20.71 seconds after permission-related rerun.
Current static/config checks	Passed	Tracked source and dependency metadata inspected; docker compose config --quiet and git diff --check passed.
Current runtime availability	Docker unavailable	Docker engine pipe was absent. Live PostgreSQL and real-model integration checks were not rerun.
Prior integration result MVP_REVIEW.md	114 passed 1 skipped 216 subtests passed	Recorded earlier with database and cached embedding inference enabled; historical evidence, not a new run today.

Saved live policy evaluation: 30 September 2026

Case	Saved evidence	Outcome
Probation leave	Retrieved pages 11, 47, 9, 47, 49; LLMResponseError.	Failed answer validation. Existing review records an uncited false fallback.
Manager-approved leave	Retrieved pages 47, 47, 10, 13, 11; LLMTimeoutError.	Generation exceeded the saved 60-second deadline.
Unsupported question	Final response retained with zero sources.	Passed fallback case according to the existing review.

The default run deliberately disables RUN_DB_TESTS, RUN_MODEL_TESTS and RUN_POLICY_TESTS. Its 18 skipped tests comprise 10 ingestion database tests, 7 retrieval database tests and 1 live-policy test; two database tests additionally require the real embedding model flag.

Interpretation
The wiring, validation and failure paths have substantial automated coverage. Today's passing test count does not prove live database health, retrieval quality across a broad corpus, model grounding, load capacity or production availability.

Evidence: tests/; current pytest output; MVP_REVIEW.md; .cache/policy-evaluation/*.json (metadata inspected without reproducing private passages).

Current gaps and constraints

These findings follow from the source and the saved evaluation evidence. They define the present boundary of the project; no application changes were made for this report.

Priority / area	Finding and impact	Next acceptance check
High: answer quality	Saved real-policy cases show missed evidence and a timeout. Citation validity alone cannot confirm factual support.	Repeat representative questions with human-reviewed expected claims, citations and fallback behavior.
High: shared access	No authentication, document ownership or tenant isolation. Any caller reaching the API can search ready documents.	Before shared use, verify that one user cannot read or select another user's documents.
High: operations	Dockerfile is empty. Compose runs only the database. No CI workflow or deployment configuration is present.	Build and run the complete service from a clean checkout with a documented release check.
Medium: ingestion recovery	Synchronous requests have no persistent job owner/lease or automatic stale-pending recovery.	Simulate a worker interruption and verify safe retry without partial ready documents.
Medium: prompt capacity	top_k can reach 20; no generation-token budgeting or context-window enforcement exists.	Measure prompt tokens and latency with the actual generation tokenizer and server settings.
Medium: search scale	Exact cosine search has no ANN index, hybrid retrieval or reranking. No search benchmark is included.	Measure corpus size, latency and retrieval recall before choosing an indexing change.
Medium: observability	Error logs and opt-in local traces exist; no metrics, request correlation or broad evaluation dashboard.	Track upload/search/generation latency and failure rates without logging private passages by default.
Product scope	No frontend, conversation memory, document list/delete/download APIs or OCR.	Agree which of these are required for the next user-facing milestone.

Smaller cleanup items: reconcile the README/example timeout values; label historical NOTES.md sections clearly; decide whether unused scaffold modules should remain; align query-size limits across search and chat. These do not change the fact that the core pipeline is implemented.

Evidence: Application routes and models; Dockerfile; repository inventory; app/services/rag.py; MVP_REVIEW.md.

Milestones and next steps

Repository history shows a progression from infrastructure to ingestion, retrieval and chat. Remaining work should be accepted against observable behavior.

Date / commit	Milestone	Current reading
23 Sep / 31f2855	Initial FastAPI setup	Foundation scaffold established.
24 Sep / 5f1d71b	Async database + embeddings	Configuration, schema and provider foundation implemented.
28 Sep / f73fad2	PDF ingestion + vector search	Document indexing and retrieval implemented.
29 Sep / 36b0b11	RAG API wiring	End-to-end application path implemented; live model checking remained.
30 Sep / c916486	Chat/model fixes	Citation checks, transaction handling and diagnostic/evaluation work reflected in current source.

Proposed order for the next milestones

Step	Work	Exit condition
1. Reliable local demo	Run services, align timeout guidance, evaluate real questions, inspect traces and tune the chosen model/prompt.	Reviewed answers capture relevant rules and exceptions, cite supporting pages and meet an agreed latency target.
2. User workflow	Define the required frontend, document management and conversation behavior.	A user can upload, choose documents, ask questions and inspect sources without manual API assembly.
3. Repeatable delivery	Complete app packaging, clean-checkout setup, CI checks and runtime dependency checks.	A documented installation/release procedure passes on a fresh environment.
4. Shared service	Add identity/ownership, capacity controls, job recovery, backup/restore and monitoring.	Access-isolation, recovery and load tests pass for an agreed supported workload.

Measure before scaling

Record retrieval recall, grounded-answer quality, fallback correctness, and upload/search/generation latency separately. No throughput target, latency SLA, coverage percentage or broad quality score is established by the current repository.

Evidence: git log; README.md; NOTES.md; MVP_REVIEW.md. Next milestones are proposals based on observed gaps.

Local operation and key terms

This is the operating sequence described by the repository. It was reviewed against source; the complete live stack was not started during this report task.

Order	Action	Expected result
1	Configure .env from the example for a new checkout; keep existing secrets. Run <code>uv sync --locked</code> .	Locked Python environment is available.
2	Start Docker Desktop, then <code>docker compose up -d db</code> .	PostgreSQL becomes healthy on localhost:5433.
3	Run <code>uv run alembic upgrade head</code> , then <code>uv run alembic current</code> .	Schema revision 7f7483ec3680 is installed.
4	Run the configured Ollama server and ensure the exact <code>LLM_MODEL</code> is available.	The chat completion endpoint can serve the selected model.
5	Start <code>uv run python run.py</code> .	API listens on localhost:8000 with a Windows-compatible event loop.
6	Open /docs; check /v1/health and /v1/ready; upload a PDF and ask a question.	API, ingestion, retrieval and generation are checked in sequence.

Useful developer entry points

Ingest: `uv run python -m scripts.ingest path/to/file.pdf`

Trace: `uv run python -m scripts.trace_rag "Your question" --document-id UUID`

Tests: `uv run pytest -W error::DeprecationWarning tests -v`

A trace records ranked chunks, messages, model settings, raw response when available, and the final result or failure type. Trace files contain document text. Normal API calls do not automatically write traces.

Plain-language glossary

Term	Meaning in this project
Chunk / embedding	A small page passage / a 384-number representation used to compare meaning.
Cosine distance / top_k	The retrieval ranking measure (lower is closer) / maximum number of passages returned.
RAG / grounding	Retrieval followed by generation / answering from the supplied evidence.
Migration / transaction	A versioned schema change / database changes that commit or roll back together.
MVP / readiness	An initial working product scope / the current database-dependency health check.

Evidence: README.md; run.py; scripts/ingest.py; scripts/trace_rag.py; migrations/env.py.

File inventory: root and infrastructure

Every tracked file is accounted for in pages 14-17. Empty package markers are listed separately from unfinished feature scaffolds.

File	Role / status
.env.example	Sample database, embedding and LLM settings; current timeout is 60 seconds.
.gitignore	Excludes secrets, virtual environment, caches, compiled files and local PDFs.
.python-version	Pins the selected Python minor version to 3.12.
pyproject.toml	Project v0.1.0, Python requirement, 13 runtime dependencies and pytest dev group.
uv.lock	Resolved dependency graph with 56 package records; root metadata matches the manifest.
README.md	Current setup, API usage, limits, tests, tracing and MVP boundary documentation.
NOTES.md	Historical development log; early unfinished-state descriptions are superseded by later code.
MVP_REVIEW.md	Prior audit, implemented fixes, recorded integration results and open live-model failures.
run.py	Local Uvicorn launcher and explicit Windows selector event loop.
compose.yaml	PostgreSQL/pgvector service, localhost binding, persistent volume and health check.
Dockerfile	Empty scaffold. Application container image is not implemented.
alembic.ini	Migration location, path handling and logging; actual DB URL is built in env.py.
infra/postgres/init/001-vector.sql	Creates vector extension when a fresh database volume initializes.
migrations/README	Generic single-database migration note.
migrations/env.py	Online/offline migrations, settings-based URL and explicit pgvector type rendering.
migrations/script.py.mako	Template for new migration scripts.
migrations/versions/7f7483ec3680_create_documents_and_document_chunks.py	Initial reversible migration creating document/chunk tables and constraints.

File inventory: API and contracts

The app directory contains the executable backend. Routes use schemas to make accepted requests and returned responses explicit.

File	Role / status
app/main.py	FastAPI factory, /v1 registration, startup database check and shutdown cleanup.
app/core/config.py	Cached application/database/embedding settings, secret handling and limits.
app/api/v1/router.py	Combines health, readiness, document and chat routers.
app/api/v1/health.py	GET liveness response.
app/api/v1/readiness.py	GET readiness response with HTTP 503 on failed database check.
app/api/v1/documents.py	Bounded upload reading, input validation, ingestion and search HTTP handlers.
app/api/v1/chat.py	Chat endpoint, request-scoped session dependency and LLM/storage error mapping.
app/schemas/health.py	Typed ok health response and message.
app/schemas/readiness.py	Typed ready/not_ready response.
app/schemas/documents.py	Filename rules, upload/search responses, document filters and ingestion batch contracts.
app/schemas/chat.py	Strict question, top_k, source metadata and chat response contracts.
app/prompts/rag.py	Grounding system prompt and canonical insufficient-context answer.

Empty Python package markers

```

app/__init__.py
app/api/__init__.py
app/api/v1/__init__.py
app/core/__init__.py
app/prompts/__init__.py
app/providers/__init__.py
app/schemas/__init__.py
app/services/__init__.py
app/storage/__init__.py

```

These nine empty `__init__.py` files define package boundaries. Their emptiness does not indicate missing feature implementation.

File inventory: processing and storage

Feature behavior lives in these implementations. Some originally planned module names remain empty; their working equivalents are identified below.

File	Role / status
app/providers/base.py	Abstract embedding interface: query, documents, dimensions and space identifier.
app/providers/documents.py	PDF envelope checks, strict extraction, limits and page-aware text structures.
app/providers/embeddings.py	Cached provider factory; enforces supported model and dimensions.
app/providers/fastembed_provider.py	Lazy local inference, thread offloading/lock and normalized vector validation.
app/providers/chat_model.py	Empty scaffold. Actual LLM client is app/services/llm.py.
app/providers/models.py	Empty scaffold. Database models live in app/storage/models.py.
app/services/document_hash.py	SHA-256 fingerprint for exact uploaded bytes.
app/services/chunking.py	BGE tokenizer loading and page-based overlapping text windows.
app/services/ingestion.py	Deduplication flow, extraction, claims, embeddings, publish and failure cleanup.
app/services/retrieval.py	Query normalization, provider/vector checks and storage search orchestration.
app/services/llm.py	Independent LLM settings, HTTPX request, deadline, completion parsing and errors.
app/services/rag.py	Short retrieval transaction, JSON prompt, generation, traces and citation validation.
app/services/chat.py	Empty scaffold. Working chat coordination is in app/services/rag.py.
app/storage/db.py	Async engine, pool/timeouts, session/transaction contexts and readiness.
app/storage/models.py	Typed documents/chunks tables, fixed embedding contract and constraints.
app/storage/documents.py	Deduplication claims, retry, atomic completion, failure cleanup and exact search SQL.
app/storage/vector_store.py	Empty scaffold. Vector query implementation is in app/storage/documents.py.

File inventory: tests and local artifacts

Tests cover parsing, contracts, storage integration, provider handling and RAG behavior. The live quality evaluation is separately enabled.

File	Role / status
scripts/ingest.py	Bounded-file ingestion CLI; supports module and direct-file invocation.
scripts/trace_rag.py	Explicit local trace command using the actual RAG pipeline.
evaluation/questions.json	Three policy/fallback cases with expected pages and human review criteria.
tests/__init__.py	Test package marker with a short module docstring.
tests/helpers.py	Generated PDF fixtures, stub embeddings, small tokenizer and Windows async runner.
tests/test_documents.py	19 tests: extraction, limits, chunking, filenames, CLI and upload statuses.
tests/test_ingestion.py	10 tests: DB claims, retries, cancellation, rollback and optional real inference.
tests/test_retrieval.py	27 tests: filters, vector checks, exact ranking, transactions and optional real retrieval.
tests/test_llm.py	20 tests: configuration, request format, response parsing, deadlines and cancellation.
tests/test_rag_service.py	18 tests: prompts, citation checks, fallback, traces and retrieval/generation wiring.
tests/test_chat_api.py	15 tests: API contract, error mapping, session cleanup and OpenAPI.
tests/test_rag.py	3 API/service integration tests with mocked retrieval and model responses.
tests/test_trace_rag.py	2 tests: retained failure evidence and redacted exception details.
tests/test_policy_live.py	1 opt-in test method covering all 3 real policy evaluation cases.

Local and generated folders

.env/: local configuration; key names checked, values omitted. **data/documents/**: ignored 28-page Q-Tickets manual (1,911,119 bytes), an input fixture. **.cache/**: FastEmbed assets, prior audit traces and policy evaluation traces; metadata reviewed selectively.

.venv/: installed third-party runtime, inspected through package metadata. **.git/**: version history and tracking metadata. **.pytest_cache/** and **__pycache__**: generated test/bytecode artifacts. **pytest-cache-files-r3kf4dgu/**: inaccessible temporary cache directory; contents not reviewed.

output/pdf/ and **tmp/pdfs/**: created by this task for the final report, editable companion, generation script, rendering dependency and visual verification files. They are documentation tooling, not application architecture.